Irembo Voice AI Intent Classification Report

18th January 2026

Code Repository: `https://github.com/Abdulraqib20/irembo-voice-ai`
Code and reproducibility: See scripts for preprocessing, training, and evaluation.

Approach

I fine-tuned a multilingual transformer, Davlan/afro-xlmr-base, for intent classification. This model was selected because it includes African language coverage and handles morphology and code-switching between Kinyarwanda and English. In production, I use a hybrid routing strategy: the transformer is primary, a rule-based classifier provides a fast baseline and secondary fallback, and a Mixtral-8x7B LLM fallback is used for low-confidence or ambiguous cases. The transformer remains the primary model because it delivers higher macro performance across all intents.

Why this fits Voice AI: Voice AI systems face unique NLP constraints that guided model selection:

- **ASR noise tolerance:** Speech-to-text introduces errors ("rendez-vous" → "rendezvous", "pasiporo" → "passport"). Transformers learn robust representations that generalize across spelling variants, unlike brittle keyword matching.
- **Code-switching:** Rwandan users naturally mix Kinyarwanda and English ("Ndashaka kureba status"). AfroXLMR's multilingual pretraining handles cross-lingual subword sharing.
- **Informal phrasing:** Spoken language differs from written text including contractions, hesitations, incomplete sentences. The model sees diverse training examples including informal patterns.
- **Latency requirements:** Voice interactions expect sub-second responses. The transformer runs in 60–100ms on CPU, acceptable for real-time use.

Data Strategy

The dataset uses three splits: 561 train, 70 validation, and 69 test samples. Preprocessing standardizes fields and produces a HuggingFace JSON format for training and evaluation.

Noise and transcription errors I retain noise to reflect real ASR conditions. Basic normalization is limited to whitespace handling to avoid destroying code-switched tokens. Future work can include ASR confidence-based filtering and spelling normalization.

Code-switching Training includes Kinyarwanda, English, and mixed utterances. The tokenizer learns cross-language subwords. Evaluation is reported per language to ensure mixed and Kinyarwanda performance remain strong.

Low-resource strategy I used the provided labeled data and preserved the given train/val/test splits for core training. In this codebase, I did not apply synthetic augmentation (no template paraphrasing, back-translation, or synonym replacement). Instead, I focused on multilingual transfer by fine-tuning AfroXLMR and added lightweight language detection plus a hybrid fallback path to improve robustness on scarce Kinyarwanda and code-switched utterances.

Data quality improvement over time Feedback loop for continuous improvement: low-confidence utterances (<0.7) flagged for human review; agent escalation captures failed utterances for labeling; retraining triggered after 100+ new samples; A/B deployment tests new versions on 10% traffic first.

Evaluation

I report accuracy, macro precision, macro recall, and macro F1. Macro metrics are chosen to weight all intents equally, which is critical for public services. Weighted metrics can hide poor performance on rare intents.

Model	Accuracy	Macro F1	Macro Precision	Macro Recall
Rule-Based Baseline	88.41%	0.8937	0.9034	0.8857
AfroXLMR base	95.65%	0.9563	0.9654	0.9526

Table 1: Overall test performance.

Per-language accuracy English 90.5%, code-switched 100.0%, Kinyarwanda 97.4%. This demonstrates strong performance on the target language and on mixed utterances.

Robustness and failure modes Systematic failure analysis reveals:

- **Confusable intents:** "requirements_information" vs. "fees_information" overlap when users ask "What do I need?" (documents or money?). Mitigation: clarifying follow-up question.
- **Out-of-scope utterances:** Users may ask unrelated questions ("What's the weather?"). The model assigns low confidence, triggering escalation.
- **Edge cases tested:** Empty input, single-word queries, very long utterances, profanity—all handled gracefully with fallback.

Confusion matrix analysis in evaluation scripts guides dataset expansion for ambiguous pairs.

MLOps and Production Considerations

Model Versioning and Deployment

A file-based model registry (`src/config/model_registry.py`) tracks versions (semantic versioning), training configs, evaluation metrics, data hash, and active version with rollback capability. Deployment uses FastAPI (`src/api/inference_api.py`) with health/readiness endpoints, single and batch classification, and three-tier fallback: Transformer → Rule-based → Groq LLM (`mixtral-8x7b-32768`).

Monitoring in Production

The monitoring module (`src/config/monitoring.py`) tracks: latency (avg/P95), confidence distribution, fallback rate, and language drift detection. Alerts trigger when low-confidence rate >20%, fallback rate >15%, P95 latency >300ms, or language distribution shifts >15%. All requests are logged as JSONL for offline analysis.

Real-Time Voice AI Pipeline Integration

The classifier integrates as: (1) ASR provides transcribed utterance, (2) language detector identifies en/rw/mixed, (3) transformer predicts intent with confidence, (4) if confidence <0.7, route to fallback, (5) map intent to backend service. The API exposes `/classify` (real-time, <100ms on GPU) and `/classify/batch` (bulk processing).

Fallback Strategies

Three-tier fallback protects citizens from misrouting: (1) Transformer (confidence ≥0.7), (2) Rule-based comparison (use higher confidence), (3) Groq LLM for edge cases. All fallback events are logged with reason for post-hoc analysis. This leverages speed (rule-based), interpretability, and semantic understanding (LLM).

Voice AI Pipeline Architecture (Bonus)

The end-to-end flow: **User Speech** → **ASR** → **Language Detection** (en/rw/mixed) → **Intent Classification** (Transformer → Rule-based → Groq LLM fallback) → **Action Router** → **Response (TTS/Text)**. Each stage logs metrics for observability.

Ethics and Public Sector Considerations

Bias and fairness risks Government services must serve all citizens equitably. Key risks and mitigations:

- **Language bias:** Kinyarwanda speakers should not receive worse service than English speakers. I evaluate per-language accuracy separately (Kinyarwanda 97.4% vs English 90.5%) and prioritize Kinyarwanda performance.
- **Intent coverage bias:** Rare intents (e.g., complaint_or_support_ticket) may be underrepresented. Macro F1 weights all intents equally, preventing majority-class dominance.

- **Demographic proxies:** Language choice may correlate with rural/urban or education level. Regular audits should check if certain user groups experience higher escalation rates.

Explainability Citizens deserve transparency: the API returns confidence scores; low confidence triggers explicit "I'm not sure" responses; each intent includes canonical examples for explanation; escalation to agents is communicated clearly ("I couldn't understand your request. Connecting you to an agent.").

Responsible AI Public sector AI requires: no silent failures (uncertain predictions escalate to humans); audit trail (all predictions logged with request ID); human oversight (system augments, not replaces, agents); data privacy (utterance logs anonymized, PII redacted).

Tradeoffs and Limitations

- **Accuracy vs. latency:** Transformer (95.65% acc, ∼60–100ms) vs. rule-based (88.4% acc, ∼16ms). I chose accuracy for citizen-facing correctness, with rule-based as fast fallback.
- **Model size vs. deployment cost:** AfroXLMR-base (278M params) requires ∼1GB memory. For edge deployment, distillation to a smaller model would trade accuracy for resource efficiency.
- **External API dependency:** Groq LLM fallback adds cost ($0.05–0.10 per 1K tokens) and latency (∼200–400ms). For cost-sensitive deployments, disable LLM tier or use local smaller LLM.
- **Dataset size:** 700 samples is small; results should be validated with real production traffic. The feedback loop addresses this over time.
- **ASR error propagation:** The classifier sees ASR output, not original speech. Severe transcription errors (e.g., "passport" → "pass port") may confuse the model. ASR confidence filtering could help.

Multilingual expansion strategy To extend beyond Kinyarwanda/English:

1. **Add language:** Collect 200+ labeled utterances in target language (e.g., French, Swahili).
2. **Update language detector:** Add marker words for new language to `enhanced_language_detector.py`.
3. **Fine-tune incrementally:** Continue training from existing checkpoint with mixed old + new data to prevent catastrophic forgetting.
4. **Evaluate separately:** Report per-language metrics to ensure new language doesn't degrade existing performance.

Conclusion

AfroXLMR-base provides a reliable, multilingual intent classifier for the Irembo Voice AI use case. It performs strongly on Kinyarwanda (97.4%) and code-switched speech (100%) and integrates seamlessly into a production pipeline with confidence-based fallbacks (rule-based + Groq Mixtral

LLM) and comprehensive monitoring. The system is designed to be fair (macro F1 metrics), transparent (logging all decisions), and resilient (three-tier fallback) for a citizen-facing government services platform.

A working demo is available via Docker containerization with FastAPI inference endpoints, demonstrating production readiness with health checks, batch processing, and real-time classification.